Tài Liệu Phân Tích Code `cmd-personal`

1. Mục đích của module này

`cmd-personal` là một chương trình command-line viết bằng C, chạy trên máy host trong Terminal. Nó có 2 vai trò chính:

1. Cung cấp một CLI engine có thể tái sử dụng: parsing input, quản lý prompt, lưu history, help, dispatch command và xuất output qua callback tùy chọn.
2. Cung cấp một demo "personal command manager": quét workspace để tìm các thư mục có `Makefile`, sau đó cho phép người dùng `build`, `run`, `clean`, `rebuild` các project đó ngay trong một shell tương tác.

Nói ngắn gọn, đây là một shell nhỏ phục vụ học tập và demo kiến trúc CLI trong C.

2. Các file chính

File	Vai trò
`new_cmd_line.h`	Header public, chứa macro cấu hình, kiểu dữ liệu, API public
`new_cmd_line.c`	Toàn bộ phần cài đặt CLI engine, built-in commands, workspace manager, tab completion và `main()` demo
`Makefile`	Build binary host-side và chạy chương trình tương tác
`out/ftel_workspace_manager`	Binary được tạo ra sau khi build
`README.md`	Hướng dẫn sử dụng nhanh cho user

3. Kiến trúc tổng thể

Code được tổ chức thành 4 lớp chính:

3.1. Lớp CLI engine

Đây là phần lõi dùng chung, chịu trách nhiệm:

- Khởi tạo context runtime.
- Nhận input từ Terminal.
- Cắt khoảng trắng đầu cuối.
- Tách input thành `argc/argv`.
- Tìm command phù hợp.
- Gọi handler tương ứng.
- Trả mã trạng thái cho caller.

Phần này có thể tái sử dụng cho project khác mà không cần giữ nguyên demo workspace manager.

3.2. Lớp built-in commands

Đây là nhóm lệnh luôn có sẵn trong shell:

- `help`
- `about`
- `version`
- `system`
- `status`
- `time`
- `echo`
- `prompt`
- `history`
- `clear`
- `exit`

Các lệnh này phục vụ thao tác shell chung, không phụ thuộc vào workspace manager.

3.3. Lớp workspace manager demo

Đây là phần mở rộng gắn vào CLI engine để biến shell thành công cụ quản lý project:

- Tìm workspace root.
- Quét thư mục có `Makefile`.

- Liệt kê project.
- Resolve tên project.
- Gọi `make`, `make run`, `make clean`.

Nhóm lệnh của lớp này là:

- `workspace`
- `projects`
- `refresh`
- `build`
- `run`
- `clean`
- `rebuild`

3.4. Lớp interactive terminal + completion

Khi chạy trong terminal thật, chương trình:

- bật raw mode,
- đọc từng phím,
- hỗ trợ `Tab` completion,
- xử lý `Backspace`,
- xử lý `Ctrl + D`,
- đảm bảo restore terminal state khi thoát hoặc nhận signal.

4. Luồng chạy tổng thể của chương trình

Luồng chạy của bản demo standalone bắt đầu từ `main()` trong `new_cmd_line.c`.

4.1. Khởi động

`main()` thực hiện các bước:

1. Tạo `new_cmd_context_t ctx`.
2. Gọi `new_cmd_line_init(&ctx, "Personal CMD Manager", "v.0.0.1")`.
3. Gọi `manager_initialize(&ctx)` để chuẩn bị workspace metadata.
4. Gắn bảng lệnh mở rộng bằng `new_cmd_line_attach_commands(&ctx, g_demo_commands)`.
5. Đổi prompt thành `workspace>`.
6. Gọi `new_cmd_line_run(&ctx)` để vào vòng lặp REPL.

4.2. Vòng lặp REPL

Trong `new_cmd_line_run()`:

- Shell in tên app và version.
- In dòng hướng dẫn `Interactive shell ready...`.
- Hiển thị prompt.
- Đọc input từ `stdin`.
- Nếu là TTY thật và có `NEW_CMD_LINE_ENABLE_MAIN`, shell dùng `read_line_with_completion()`.
- Nếu không, shell fallback sang `fgets()`.
- Sau khi đọc xong, shell chuyển input sang `new_cmd_line_process()`.

4.3. Xử lý một dòng lệnh

`new_cmd_line_process()` là hàm quan trọng nhất của lõi CLI.

Nó làm lần lượt:

1. `copy_trimmed_input()` để bỏ khoảng trắng đầu/cuối.
2. `push_history()` để lưu history.
3. `tokenize_input()` để tạo `argc/argv`.
4. `find_command()` để tìm command.
5. Gọi `handler` của command.
6. Trả về `new_cmd_status_t`.

Nếu có lỗi, hàm này in message tương ứng cho user:

- input quá dài,
- quá nhiều tham số,
- sai cú pháp quote,
- command không tồn tại,
- handler trả lỗi.

5. Phân tích `new_cmd_line.h`

Header này định nghĩa toàn bộ public contract của module.

5.1. Macro cấu hình

Các macro có thể override trước khi include header:

Macro	Giá trị mặc định	Ý nghĩa
`NEW_CMD_MAX_INPUT_LEN`	`256`	Độ dài tối đa của một dòng lệnh
`NEW_CMD_MAX_ARGS`	`10`	Số lượng argument tối đa
`NEW_CMD_HISTORY_DEPTH`	`8`	Số dòng history giữ trong RAM
`NEW_CMD_MAX_PROMPT_LEN`	`32`	Độ dài tối đa của prompt
`NEW_CMD_DEFAULT_PROMPT`	`"ftel> "`	Prompt mặc định ban đầu

5.2. `new_cmd_status_t`

Enum này mô tả trạng thái xử lý của shell:

- `NEW_CMD_STATUS_OK`
- `NEW_CMD_STATUS_EMPTY_INPUT`
- `NEW_CMD_STATUS_NULL_CONTEXT`
- `NEW_CMD_STATUS_INPUT_TOO_LONG`
- `NEW_CMD_STATUS_TOO_MANY_ARGS`
- `NEW_CMD_STATUS_PARSE_ERROR`
- `NEW_CMD_STATUS_NOT_FOUND`
- `NEW_CMD_STATUS_HANDLER_ERROR`

5.3. Kiểu callback và command entry

- `new_cmd_writer_t`: callback để chuyển hướng output sang nơi khác thay vì `stdout`.
- `new_cmd_handler_t`: prototype chuẩn cho mọi command handler.
- `new_cmd_command_t`: mô tả một lệnh với `name`, `handler`, `usage`, `description`.

5.4. `new_cmd_context_t`

Đây là runtime state của shell. Các field quan trọng:

- `app_name`, `version`: thông tin nhận diện ứng dụng.
- `external_commands`: bảng lệnh mở rộng ngoài built-in.
- `writer`, `writer_user_data`: đầu ra tùy chỉnh.
- `started_at`: thời điểm shell khởi động.
- `history_total`: tổng số lệnh đã nhập từ đầu phiên.

- `running`: cờ để giữ hoặc dừng REPL.
- `history_count`, `history_head`: quản lý history dạng vòng tròn.
- `prompt`: chuỗi prompt hiện tại.
- `history[][]`: vùng nhớ chứa các lệnh trước đó.

5.5. Public API

Hàm	Vai trò
`new_cmd_line_init()`	Khởi tạo context
`new_cmd_line_set_writer()`	Đổi output writer
`new_cmd_line_set_prompt()`	Đổi prompt
`new_cmd_line_attach_commands()`	Gắn thêm bảng lệnh mở rộng
`new_cmd_line_stop()`	Yêu cầu shell dừng
`new_cmd_line_run()`	Chạy REPL
`new_cmd_line_printf()`	In formatted output
`new_cmd_line_process()`	Xử lý một dòng lệnh độc lập
`new_cmd_line_status_string()`	Chuyển enum status sang chuỗi

6. Phân tích `new_cmd_line.c`

6.1. Nhóm helper cơ bản

Các helper đầu file xử lý những việc nền tảng:

- `default_writer()`: fallback output sang `stdout`.
- `write_text()`: route output qua writer đã cấu hình.
- `command_name_equals()`: so sánh tên lệnh không phân biệt hoa thường.
- `count_commands()`: đếm số entry trong command table.
- `find_in_table()`: tìm command trong một bảng lệnh.
- `find_command()`: tìm command theo thứ tự `external_commands` trước, built-in sau.

Một chi tiết đáng chú ý:

- `find_command()` bỏ qua ký tự `*` ở đầu command.
- Vì vậy `*Build project cmd-personal` vẫn được map về handler `build`.

6.2. Xử lý prompt, history và input

Các hàm chính:

- `copy_prompt()`: sao chép prompt có kiểm soát độ dài.
- `copy_trimmed_input()`: bỏ khoảng trắng đầu/cuối.
- `push_history()`: lưu lệnh mới vào history vòng tròn.
- `clear_history()`: reset history.
- `tokenize_input()`: parse input thành `argv[]`.
- `join_arguments()`: ghép argument lại thành chuỗi.

`tokenize_input()` hỗ trợ:

- chuỗi trong `'single quote'`,
- chuỗi trong `"double quote"`,
- ký tự escape bằng `\`.

Nếu người dùng nhập sai dấu nháy hoặc quá nhiều argument, hàm sẽ trả status lỗi.

6.3. Xử lý thời gian và thông tin hệ thống

Nhóm hàm này phục vụ các lệnh `status`, `time`, `system`:

- `format_time_value()`
- `format_uptime()`
- `compiler_name()`
- `platform_name()`
- `c_standard_name()`

Chúng giúp shell in được:

- thời gian local,
- uptime,
- compiler hiện tại,
- platform hiện tại,
- chuẩn C đang dùng.

6.4. Hệ thống help

Hai helper chính:

- `print_command_summary()`
- `print_command_help()`

Chúng được dùng bởi lệnh `help` để:

- liệt kê toàn bộ lệnh,
- hoặc in mô tả chi tiết cho một lệnh cụ thể.

6.5. Hàm public quan trọng

`new_cmd_line_init()`

Khởi tạo context về trạng thái mặc định:

- zero toàn bộ struct,
- đặt `app_name`,
- đặt `version`,
- ghi lại `started_at`,
- bật `running`,
- gán prompt mặc định.

`new_cmd_line_set_writer()`

Cho phép nhúng module này vào môi trường khác ngoài terminal thông thường, ví dụ:

- UART console,
- GUI log viewer,
- test harness,
- bộ ghi log riêng.

`new_cmd_line_process()`

Đây là tâm điểm của parser/dispatcher.

Nó tách hẳn việc xử lý một dòng lệnh ra khỏi REPL, nên module này có thể:

- dùng ở interactive shell,
- dùng ở unit test,
- dùng với input đến từ script,
- dùng như backend cho một front-end khác.

`new_cmd_line_run()`

Đây là REPL blocking loop:

- in banner,
- đọc lệnh,
- phát hiện EOF,
- chặn input quá dài,
- chuyển lệnh sang parser.

7. Built-in commands đang có

Lệnh	Chức năng
`help`	Liệt kê tất cả lệnh hoặc chi tiết từng lệnh
`about`	In mục đích và tính năng chính của shell
`version`	In tên app, version, ngày giờ build
`system`	In compiler, platform, chuẩn C và số lượng command
`status`	In runtime state của shell
`time`	In local time và uptime
`echo`	In lại chuỗi đầu vào
`prompt`	Xem hoặc đổi prompt hiện tại
`history`	Xem hoặc xóa history
`clear`	Xóa màn hình bằng ANSI escape code
`exit`	Thoát khỏi shell

Một vài chi tiết nhỏ nhưng hữu ích:

- `prompt` tự thêm dấu cách ở cuối nếu cần.
- `history` dùng buffer vòng tròn nên chỉ giữ `NEW_CMD_HISTORY_DEPTH` lệnh gần nhất.
- `clear` chỉ hoạt động đúng trên terminal hỗ trợ ANSI.

8. Workspace manager demo

Phần này chỉ được biên dịch khi bật `NEW_CMD_LINE_ENABLE_MAIN`.

8.1. Dữ liệu chính

Có 2 struct quan trọng:

- `new_cmd_project_t` lưu `relative_path` và `short_name` của từng project.
- `new_cmd_manager_t` lưu `workspace_root`, danh sách project, số project hiện có, số lần refresh và cờ `truncated`.

Ngoài ra còn có một biến toàn cục:

- `g_manager`

Biến này giữ cache project cho suốt vòng đời chương trình.

8.2. Cách xác định workspace root

`manager_find_workspace_root()`:

- lấy thư mục hiện tại bằng `getcwd()`,
- đi ngược dần lên parent directory,
- dừng khi thấy thư mục `.git`.

Nếu không tìm thấy `.git`, hàm fallback về thư mục hiện tại.

Điều này giúp shell chạy từ thư mục con vẫn tìm được workspace root ổn định.

8.3. Cách quét project

Luồng discovery:

1. `manager_refresh_projects()` reset cache cũ.
2. `manager_discover_recursive()` quét đệ quy.
3. Thư mục nào chứa `Makefile` hoặc `makefile` sẽ được xem là một project.

4. `manager_add_project()` thêm project mới vào cache.

Các thư mục bị bỏ qua:

- thư mục ẩn bắt đầu bằng `.`,
- `.`, `..`,
- `out`,
- `build`.

Giới hạn hiện tại:

- tối đa `64` project,
- độ sâu quét tối đa `8` cấp.

8.4. Cách resolve tên project

`manager_find_project()` resolve theo thứ tự:

1. so khớp tuyệt đối với `relative_path`,
2. nếu không có thì so khớp với `short_name`.

Nếu `short_name` bị trùng ở nhiều nơi:

- shell báo lỗi ambiguous,
- user phải dùng full relative path.

8.5. Cách chạy `make`

Tất cả các lệnh `build`, `run`, `clean`, `rebuild` cuối cùng đều đi về `manager_execute_make()`.

Hàm này:

- ghép ra `project_path`,
- kiểm tra path tồn tại,
- in lệnh sẽ chạy,
- `fork()` tiến trình con,
- gọi `execvp("make", args)` trong child,
- `waitpid()` ở parent,
- báo thành công hoặc thất bại cho user.

Ưu điểm của cách này:

- output thật của `make` được stream thẳng ra terminal,
- không cần tự viết lớp thu log trung gian,
- trải nghiệm gần giống chạy `make` trực tiếp.

8.6. Các lệnh manager

Lệnh	Chức năng
`workspace`	In workspace root, số project và số lần refresh
`projects`	Liệt kê project đã tìm thấy
`refresh`	Quét lại workspace
`build <name>`	Chạy `make`
`run <name>`	Chạy `make run`
`clean <name>`	Chạy `make clean`
`rebuild <name>`	Chạy `make clean` rồi `make`

Nhóm lệnh `build/run/clean/rebuild` chấp nhận cả 2 cú pháp:

```
build cmd-personal
build project cmd-personal
```

9. Interactive terminal và tab completion

Phần terminal tương tác được hiện thực bởi các hàm:

- `interactive_restore_terminal()`
- `interactive_signal_handler()`
- `interactive_install_handlers()`
- `interactive_enable_raw_mode()`
- `completion_parse_line()`
- `completion_collect_matches()`
- `completion_apply_candidate()`
- `completion_print_matches()`
- `read_line_with_completion()`

9.1. Raw mode

Khi chạy trong terminal thật:

- shell tắt `ICANON`,
- tắt `ECHO`,
- đọc từng ký tự,
- tự xử lý việc hiển thị.

9.2. Completion hỗ trợ những gì

Shell hỗ trợ completion cho:

- token đầu tiên là command,
- command sau `help`,
- tên project sau `build`, `run`, `clean`, `rebuild`.

Nếu có đúng 1 match:

- shell tự hoàn thành.

Nếu có nhiều match:

- shell in danh sách candidate.

Nếu các match có chung prefix dài hơn phần đã gõ:

- shell tự điền thêm prefix chung.

9.3. Xử lý phím đặc biệt

- `Enter`: kết thúc dòng lệnh.
- `Backspace`: xóa ký tự trước đó.
- `Tab`: trigger completion.
- `Ctrl + D`: thoát nếu dòng hiện tại đang trống.

9.4. An toàn khi thoát

Code có cài signal handler cho:

- `SIGINT`
- `SIGTERM`

- `SIGHUP`

Mục tiêu là khôi phục terminal state trước khi chương trình thoát, tránh để terminal bị kẹt ở raw mode.

10. Makefile đang làm gì

`Makefile` rất gọn và chỉ phục vụ bản demo host-side.

Target chính

Target	Vai trò
`all`	Mặc định, trỏ sang `build`
`build`	Biên dịch binary
`run`	Chạy binary sau khi build
`clean`	Xóa thư mục `out`

Cách build

Binary được build bằng:

```
gcc -std=c11 -Wall -Wextra -pedantic new_cmd_line.c -DNEW_CMD_LINE_ENABLE_MAIN -o ou
```

Ý nghĩa:

- bật C11,
- bật warning tương đối chặt,
- bật macro `NEW_CMD_LINE_ENABLE_MAIN`,
- build cả phần demo workspace manager và `main()`.

11. Những điểm đáng chú ý khi đọc code

Điểm tốt

- Tách khá rõ public API và implementation.

- `new_cmd_line_process()` đủ độc lập để test riêng.
- Có cơ chế attach external commands nên dễ mở rộng.
- Có writer callback để đổi đầu ra.
- Có completion cơ bản nhưng đủ hữu ích.
- Cách gọi `make` bằng `fork + execvp` khá sạch.

Giới hạn hiện tại

- Chưa hỗ trợ pipe, redirect, environment variable expansion.
- Chưa có autocompletion cho mọi kiểu command.
- History chỉ nằm trong RAM, không persist xuống file.
- Số lượng args, prompt, history và project đều có giới hạn compile-time.
- Discovery chỉ dựa vào việc có `Makefile`.
- Binary tên là `ftel_workspace_manager`, nhưng tên app hiển thị hiện tại là `Personal CMD Manager`.

Hành vi cần để ý

Trong chính thư mục `cmd-personal`, target `run` của `Makefile` chạy lại binary manager. Vì vậy:

```
run cmd-personal
```

sẽ dẫn đến việc shell gọi `make run`, và `make run` lại mở một phiên manager mới.

12. Cách mở rộng module này

Nếu muốn tái sử dụng module cho project khác, hướng mở rộng hợp lý là:

1. Giữ `new_cmd_line.h` và phần lõi CLI engine.
2. Viết bảng `new_cmd_command_t` mới cho project của bạn.
3. Gọi `new_cmd_line_attach_commands()` để gắn command mới.
4. Tùy chọn đổi output bằng `new_cmd_line_set_writer()`.
5. Tùy chọn override các macro cấu hình trong header.

Ví dụ các hướng mở rộng:

- shell cho UART trên embedded board,
- shell cấu hình thiết bị,
- shell test command cho firmware,

- shell quản lý nhiều tool build khác ngoài `make`.

13. Tóm tắt một câu

`cmd-personal` là một personal command manager viết bằng C, xây trên một CLI engine tái sử dụng được, có parser, history, prompt, help, tab completion và một lớp workspace manager để quét project rồi chạy `make` trực tiếp từ Terminal.